

Learn Low-Level Design in 25 Days: Day 12—Error Handling and Exception Design in LLD

Ever had a production bug that crashed your system because of one missed exception?

TechPreneur

Published October 24, 2025

Updated October 25, 2025

Free: No

Read on Freedium · original

Contents

The Core Problem: "When Things Go Wrong…"	0
Step 1: Define Your Exception Hierarchy	0
Step 2: Handle Errors Where They Belong	0
Step 3: Fail Fast, Fail Loud (Then Recover Gracefully)	0
Step 4: Logging, Retrying, and Alerting	0
Visual Summary	0
Try It Yourself	0
Key Takeaways	0
What's Next	0

Maybe a file didn't exist, an API returned null, or a division-by-zero slipped through — and suddenly your service was down. Most of us have been there. The frustrating part? The code looked fine, but one unhandled edge case brought the system to its knees.

Welcome to **Day 12 of the Learn Low-Level Design in 25 Days** series. Today, we're tackling something every real-world system relies on but most engineers overlook during design — **error handling and exception design**.



By the end of this lesson, you'll understand how to:

- Design classes and methods that fail gracefully, not catastrophically.
- Use exceptions as part of your design vocabulary, not just an afterthought.
- Build predictable, resilient systems — like how Netflix keeps streaming even when a subsystem fails.

The Core Problem: "When Things Go Wrong..."

Software doesn't fail when everything goes right. It fails when something unexpected happens — a null pointer, a network timeout, or an invalid user input.

Error handling exists to answer one simple question:

> "When something goes wrong, who's responsible for handling it — and how?

Let's look at this from an LLD lens.

Imagine we're designing a **PaymentService** for an e-commerce app:

```
class PaymentService {
    void processPayment(Order order) {
        // Step 1: deduct amount
        // Step 2: notify payment gateway
        // Step 3: update order status
    }
}
```

Looks clean. But what if:

- The payment gateway times out?
- The database update fails?
- The card details are invalid?

Without a structured approach to error handling, this will turn into a nest of `try-catch` blocks scattered everywhere. Worse, we might lose information about what actually went wrong.

Step 1: Define Your Exception Hierarchy

Just like you design your classes, you should **design your exceptions**.

Think of exceptions as your system's "language of failure." A clean hierarchy makes debugging and handling errors much easier.

Example:

```
class PaymentException extends Exception {}
class InvalidCardException extends PaymentException {}
class PaymentGatewayTimeoutException extends PaymentException {}
class InsufficientBalanceException extends PaymentException {}
```

Now, your `PaymentService` can throw these exceptions clearly and meaningfully:

```
class PaymentService {
    void processPayment(Order order) throws PaymentException {
        if (!cardValidator.isValid(order.getCard())) {
            throw new InvalidCardException();
        }

        if (!gateway.charge(order)) {
            throw new PaymentGatewayTimeoutException();
        }

        if (!wallet.hasSufficientBalance(order)) {
            throw new InsufficientBalanceException();
        }
    }
}
```

Why it matters: Instead of returning vague error codes or generic exceptions, we're communicating intent. Higher layers (like a controller or API handler) can now respond accordingly — retry, show user message, or log the issue.

Step 2: Handle Errors Where They Belong

A key LLD rule:

> **Handle errors at the right abstraction level.**

- **Low-level layers (e.g., network, file I/O):** Detect and throw detailed exceptions.
- **Mid-level layers (e.g., service logic):** Convert or wrap exceptions into domain-specific ones.
- **High-level layers (e.g., API, UI):** Handle exceptions gracefully and inform the user.

Example flow:

```
[Network Layer]
  ↓  (throws IOException)
[PaymentGatewayAdapter]
  ↓  (wraps as PaymentGatewayTimeoutException)
[PaymentService]
  ↓  (bubbles up)
[Controller]
  ↓  (shows “Payment failed. Try again.”)
```

This layering keeps your code clean, predictable, and debuggable.

Step 3: Fail Fast, Fail Loud (Then Recover Gracefully)

A mature design **detects errors early** and **recovers at boundaries**.

Let's say your system receives a malformed request. Should you let it flow through all layers and fail at the database? Absolutely not.

Instead:

- Validate early → Throw fast
- Catch at boundary → Handle gracefully

```
public Response handlePaymentRequest(Request req) {
    try {
        paymentService.processPayment(req.toOrder());
        return new Response("Payment Successful");
    } catch (InvalidCardException e) {
        return new Response("Invalid card details");
    } catch (InsufficientBalanceException e) {
        return new Response("Insufficient balance");
    } catch (PaymentException e) {
        return new Response("Payment failed. Try again later.");
    }
}
```

This way, internal failures don't leak into your system's public surface.

Step 4: Logging, Retrying, and Alerting

Exception design isn't just about catching — it's about **visibility**. When something fails, you want to know **where, why, and how often**.

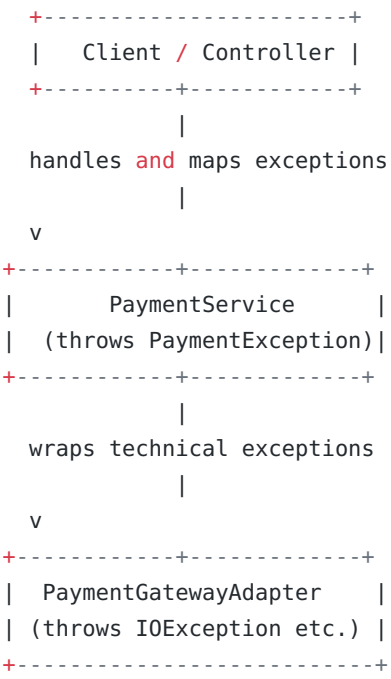
Best practices:

- **Log exceptions with context:** what input caused it, which service, and user ID.
- **Retry intelligently:** retry transient errors (like timeouts), but not logic errors.
- **Alert critical failures:** notify engineering when repeated failures occur.

For example:

```
try {
    gateway.charge(order);
} catch (TimeoutException e) {
    logger.warn("Payment timeout for order {}", order.getId());
    retryService.schedule(order);
}
```

Visual Summary



Each layer has a clear responsibility for what to do when things go wrong.

Try It Yourself

Design a FileStorageService that reads/writes files.

- Define your own exception hierarchy (`FileStorageException` , `FileNotFoundException` , `StorageFullException`).
- Decide at which layer each exception should be thrown and caught.

Look at your last project:

- Where were exceptions handled?
- Were they caught too low (swallowed errors) or too high (leaked stack traces)?

Key Takeaways

- **Exceptions are part of your design, not an afterthought.**
- **Create a clear exception hierarchy** that reflects your domain.
- **Handle errors at the right layer** — not too early, not too late.
- **Fail fast, recover gracefully, and log everything.**
- **Never swallow exceptions silently** — that's how bugs hide.

What's Next

Tomorrow, we'll **design a Logging Framework (LLD in Action)** — the perfect follow-up to exception handling. You'll learn how real systems structure logging modules, severity levels, and log pipelines.

Follow the full series here: Learn Low-Level Design in 25 Days — A Practical Bootcamp for Devs (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8)